

Session 3 · Deck 3.A3

Euler Equations, Time Iteration, and What Is Still Missing

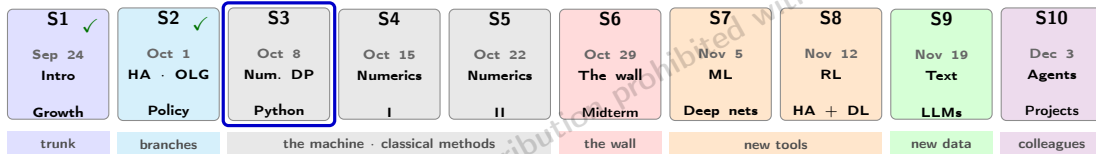
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.A3 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 3.A1 what it means to solve a model · 3.A2 value function iteration on a grid · 3.A3 Euler equations and time iteration · 3.A4 the worked example, end to end.

One Model, Two Methods, One Judge

Same model, same grid, same shock, same closed form to grade against, and both policies graded on the same object: $\max |c/c^* - 1|$.

	VFI	Time iteration
Iterations	272	17
Wall time	0.03 s	0.69 s
Max policy error	5.4×10^{-3}	6.7×10^{-6}
$\max \log_{10} \mathcal{E}$	-1.8	-3.8

Sixteen times fewer iterations and eight hundred times more accurate — and twenty times slower.

Every one of those four numbers is explained by *one* design change: iterate on the policy and solve an equation, instead of iterating on the value and searching for a maximum.

Why Leave the Value Function at All?

Three reasons, and the first two are visible in 3.A2's staircase:

1. **The policy is the object you want.** VFI drives v down to whatever tolerance you ask for, and then hands back a policy good to two digits. You paid for precision in the wrong function.
2. **The policy is flatter.** v inherits the curvature of u ; g is close to linear in this model. A flatter function is cheaper to represent on a grid.
3. **No maximization.** Each point needs the solution of one scalar equation, not a search over N candidates.

Adapted from QM Lec. 7, "From VFI to Euler Equation Methods".

What you give up

The Euler equation requires **differentiability** and an **interior** optimum. VFI needs neither — it will happily solve a problem with discrete choices, kinks, or a binding constraint, which is exactly where the Euler road ends.

So this is not a replacement. It is a second instrument, with different failure modes — which is what makes the comparison worth something.

The Euler Equation Is Also a Functional Equation

From the first-order and envelope conditions of 1.B2,

$$u'(c_t) = \beta \mathbb{E}_t [u'(c_{t+1}) R(k_{t+1}, z_{t+1})],$$

with the gross return $R(k', z') = \alpha z' k'^{\alpha-1} + 1 - \delta$. Write the policy as $k' = g(k, z)$ and substitute. Consumption today is $c = zk^\alpha + (1 - \delta)k - g(k, z)$, and consumption tomorrow uses g again:

$$u'(zf(k) - g) = \beta \mathbb{E} [u'(z'f(g) - g(g, z')) R(g, z')].$$

One equation, one unknown — and the unknown is a function that appears composed with itself.

The same two problems

Represent g finitely: values on a grid plus linear interpolation, exactly as before.

Kill the residual: but now by *root-finding*, not by maximizing.

Decisions 1, 2, 3 and 5 of 3.A1 are unchanged. Only decisions 4 and 6 — how you handle the choice, and what you iterate on — have moved.

Derivation adapted from QM Lec. 7, “Deriving Euler Equations” and “Policy Function Representation”.

Time Iteration: The Operator

Given a guess g_n for *tomorrow's* policy, define $g_{n+1} = Kg_n$ pointwise: at each (k, z) , find the k' that solves

$$u'(zf(k) - k') = \beta \mathbb{E} \left[u'(z'f(k') - g_n(k', z')) R(k', z') \right].$$

- The left side is **increasing** in k' : saving more costs more today.
- The right side is **decreasing** in k' under standard conditions.
- So they cross **once**. The root exists and is unique — before any numerical method is chosen.

Read the name literally

g_n is the policy used *one period later*. Iterating K is solving a finite-horizon problem of growing length — “time” iteration.

Which also tells you where to start: $g_0 \equiv 0$ (eat everything) is the last period of life, and the iterates then increase.

Operator and monotonicity argument adapted from QM Lec. 7, “Time Iteration”, and AE Lec. 1, “The Greenwood–Huffman Operator”.

Time Iteration: The Algorithm

1. Grid $\{k_i\}$, chain (z_j, Π) ; guess c_0 (or g_0).
2. For every (k_i, z_j) : solve the scalar equation for c , evaluating c_n off the grid by interpolation.
3. Stop when $\|c_{n+1} - c_n\|_\infty < \text{tol}$.

On our grid that is $200 \times 2 = 400$ scalar root-finds per sweep, and **6,800** in total across 17 sweeps.

```
def resid(c, y, j):  
    kp = y - c  
    rhs = 0.0  
    for m in range(nz):  
        cp = np.interp(kp, k, cn[:, m])  
        R = alpha*z[m]*kp**(alpha-1)  
        rhs += Pi[j, m] * R / cp  
    return 1/c - beta*rhs  
  
c_new[i, j] = brentq(  
    resid, 1e-10, y - 1e-10, args=(y, j))
```

Adapted from QM Lec. 7, "Time Iteration: Algorithm"; the brentq call follows QM Lab 7.

The Black Box in the Middle

brentq is handed a bracket $[a, b]$ on which the residual changes sign, and returns the root. Today we take that on trust, and we are entitled to: we *proved*, when we wrote the operator down, that the residual is increasing on one side, decreasing on the other, and crosses zero exactly once, so a bracketing method cannot fail.

That is the whole reason this course does theory before numerics. **The existence-and-uniqueness argument is what licenses the black box.**

What is inside it

Bisection is safe but slow; Newton is fast but needs a derivative and can fly off; the secant method splits the difference. Brent's method combines all three and keeps the bracket.

We build each of them, and their convergence rates, **next week**.

Until then the rule is: a black box is acceptable exactly when you can state the condition under which it is guaranteed to work.

Root-finding options listed in QM Lec. 7, "Time Iteration: Implementation Details".

Convergence Without a Contraction

VFI converged because T is a contraction — Blackwell gave us monotonicity *and* discounting, and Banach did the rest. K has no Bellman equation behind it, so that argument is unavailable.

Two repairs exist, and they are genuinely different:

- **Coleman (1990):** in the standard optimal-growth setting, K is a contraction in a suitable norm. Same guarantee, same geometric rate.
- **Greenwood–Huffman (1995):** in economies with externalities or distortions — where no planner's problem exists — K need not contract, but it is **monotone** and bounded, and that is enough.

Why the second one matters

Taxes, externalities and market power break the welfare theorems. There is then *no* value function to iterate on — but the equilibrium Euler equation still holds, and K still converges.

This is the road that survives into the models of Session 2. It is why time iteration, not VFI, is the backbone of heterogeneous-agent and optimal-policy computation.

Coleman (1990); Greenwood & Huffman (1995). Adapted from QM Lec. 7, "Convergence: The Key Question".

Monotonicity Instead of Contraction

Start at $H^0(K, z) = 0$ and define $H^{j+1}(K, z) = x$ as the solution of

$$u'(F(K, K, z) - x) = \beta \mathbb{E} \left[u'(F(x, x, z') - H^j(x, z')) \cdot F_1(x, x, z') \right],$$

where the second argument of F is the *aggregate* capital the household takes as given. The argument runs in three steps:

1. $H^{j+1} \geq H^j$ whenever $H^j \geq H^{j-1}$: the operator is **monotone**.
2. $H^j(K, z) \leq F(K, K, z)$: the sequence is **bounded** by feasibility.
3. A monotone bounded sequence converges; equicontinuity makes the limit continuous.

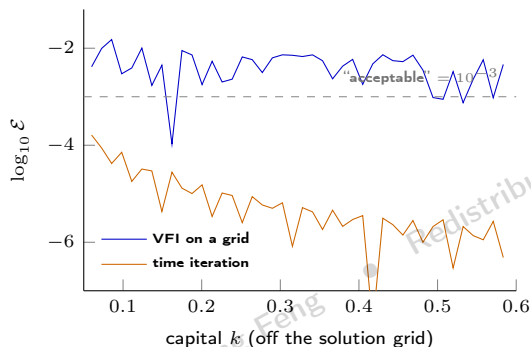
Then check it is an equilibrium

Convergence is not the end. You must still verify that the limit satisfies the *equilibrium* Euler equation once the consistency condition $k = K$ is imposed.

Practical consequence: start from $g_0 \equiv 0$ and **watch the iterates increase**. If they ever fall, the monotonicity hypothesis is violated somewhere and the guarantee is gone.

Grading Both, on Points Neither One Solved

The unit-free residual of 3.A1, at 997 capital levels chosen to fall *between* the solution grid points.



	VFI	TI
mean $\log_{10} \mathcal{E}$	-2.40	-4.92
worst $\log_{10} \mathcal{E}$	-1.78	-3.75
worst residual	1.7×10^{-2}	1.8×10^{-4}
true policy error	5.4×10^{-3}	6.7×10^{-6}
ratio	$3.1 \times$	$27 \times$

Santos (2000), checked: the residual exceeds the error it certifies, for both methods.

VFI's worst case is a **1.7% consumption mistake**, a decade worse than 3.A1's "acceptable" line.

VFI or Time Iteration — and Why You Write Both

	Value function iteration	Time iteration
Iterates on	the value v	the policy g
Inner step	a maximization	a root-find
Converges because	T contracts (Blackwell)	K contracts (Coleman) or is monotone (Greenwood–Huffman)
Handles	kinks, discrete choice, binding constraints	smooth interior problems, distorted economies
Accuracy here	$\log_{10} \mathcal{E} = -1.8$	$\log_{10} \mathcal{E} = -3.8$

Away from this benchmark there is **no closed form to grade against**. Two methods that rest on different theorems, agreeing to four digits, is then the strongest evidence available — which is why cross-method agreement sits on 1.B2's verification checklist, and why the answer to "which one?" is usually "both, once".

Comparison adapted from QM Lec. 7, "VFI vs. Time Iteration" and "When to Use Each Method"; accuracy columns measured here.

Where Both Methods Break

Everything today rested on **one continuous state and one small chain**. Four things break it, and you have already met three of them:

- **A second continuous state.** The grid becomes N^2 and a VFI sweep N^4 . 1.B2 counted where that ends: ten dimensions at a hundred points is 10^{20} grid points.
- **A binding constraint.** $a' \geq -\underline{a}$ turns the Euler equation into a complementarity condition. The root-find of this deck simply has no root.
- **A distribution as a state.** In 2.B1 the state included Γ — an infinite-dimensional object. There is no grid for it.
- **Patience.** At a quarterly $\beta = 0.99$, VFI needs 1,379 sweeps and the error amplifier is 99, not 19.

Not one of these is exotic. The first three are the standard models of Session 2 — Aiyagari, Krusell–Smith, life-cycle — and the fourth is the standard calendar.

So the honest summary of today is: **we can solve the trunk, and only the trunk**. The next two sessions are the tools that get us onto the branches.

The Techniques You Need, and When We Build Them

Technique	Which of today's gaps it closes	
Root-finding	opens the brentq box: bisection, secant, Newton, Brent, and their rates	S4
Optimization	replaces the grid search that made the staircase	S4
Differentiation	Newton needs derivatives; finite differences, then automatic differentiation	S4
Approximation & interpolation	better than straight lines, without losing shape	S4
Quadrature & AR(1) discretization	where Π actually comes from	S4
Acceleration	cuts the 272 sweeps and the $O(N^2)$ inner loop	S4
Perturbation	a local expansion that ignores the grid entirely	S5
Projection	a global basis instead of N separate values	S5
Nested fixed points	Aiyagari solved end to end	S6

Nothing on this list is decoration. **Every row was put there by a specific failure you watched happen this morning.**

Takeaways

- The Euler equation is a functional equation too. Iterating on the **policy** means solving one scalar equation per state instead of searching for a maximum.
- The economics does the numerical work: the residual is increasing on one side and decreasing on the other, so the root exists, is unique, and a bracketing solver **cannot** fail. That argument is what licenses the black box.
- Convergence has two proofs, and they are not interchangeable. **Coleman** gives a contraction in the planner's world; **Greenwood–Huffman** gives monotone convergence where no planner exists — taxes, externalities, distortions.
- Measured on the same grid, and graded on the same object: 17 sweeps against 272, consumption error 6.7×10^{-6} against 5.4×10^{-3} , $\log_{10} \mathcal{E}$ of -3.8 against -1.8 . Fewer iterations, better answers, more wall time — each sweep does 400 root-finds.
- Both methods stop at **one continuous state**. A second state, a binding constraint, or a distribution as a state, and today's toolkit is finished.

Bridge: After the Break, the Machine Itself

Every claim this morning was a measurement, and every measurement came out of twenty to forty lines of Python. That is not incidental to the argument — it *is* the argument. A method you cannot implement is a method you cannot check.

So Block B is the other half of the bridge from paper to machine: the environment, the language, and the one idea that turns an algorithm on a whiteboard into an array operation that runs in milliseconds.

After the break (Block B)

3.B1 Your toolkit: Python, environments, notebooks, and version control.

3.B2 From loops to arrays — we rebuild this morning's solver as a class, and make it 65 times faster without changing a single number.

3.B3 PyTorch from the ground up, and how to work with an AI assistant without outsourcing your judgment.